

**E se, em vez de perguntar pra uma árvore só,
perguntássemos pra várias e deixássemos
elas votarem?**



O que é Random Forest

É literalmente isso: várias árvores de decisão, cada uma vendo uma amostra ligeiramente diferente dos dados, votando juntas pela maioria.

Random porque cada árvore é treinada com aleatoriedade — tanto nos dados quanto nas features disponíveis. Forest porque são várias árvores juntas.

A ideia central: uma árvore só pode 'decorar' peculiaridades do seu recorte de dados — várias árvores, promediadas, cancelam esse ruído.



Vamos ver na prática

```
aranha = X.iloc[[df[df['nome']=='Aranha'].index[0]]]

rng = np.random.default_rng(0)
previsoes = []
for i in range(15):
    idx = rng.choice(len(X), size=len(X), replace=True)
    arvore = DecisionTreeClassifier(max_depth=3, random_state=i)
    arvore.fit(X.iloc[idx], y.iloc[idx])
    previsoes.append(bool(arvore.predict(aranha)[0]))

print(previsoes)
```



Discordância genína

Rodando 15 árvores diferentes (cada uma vendo uma amostra aleatória dos 85 animais), pedimos a todas pra classificar a mesma aranha.

Resultado: 10 disseram 'tem pelo', 5 disseram 'não tem' — dependendo de quais animais calharam na amostra de cada árvore, a resposta muda.

Não é erro de código — é a instabilidade real de uma árvore única, na cara.



Como o Random Forest resolve isso

Em vez de confiar na opinião de 1 árvore, o Random Forest treina centenas delas e agrega os votos — a probabilidade final é a proporção de árvores que votaram em cada classe.

Isso transforma um 'sim ou não' arriscado numa probabilidade calibrada, que reflete o quanto o modelo está genuinamente incerto.

Mais estável que qualquer árvore individual, porque erros aleatórios de árvores diferentes tendem a se cancelar.



Vamos ver na prática

```
floresta = RandomForestClassifier(n_estimators=100, random_state=42)
floresta.fit(X, y)

prob = floresta.predict_proba(aranha)
print(f"Probabilidade de ter pelo: {prob[0][1]:.1%}")
```



De onde vem a diversidade entre as árvores

Bootstrap: cada árvore é treinada numa amostra aleatória dos animais (com reposição) — não vê exatamente os mesmos 85 que as outras.

max_features: em cada divisão, cada árvore só pode escolher entre um subconjunto aleatório das colunas, não todas.

As duas fontes de aleatoriedade juntas garantem que as árvores sejam genuinamente diferentes entre si — senão, votar não adiantaria nada.



O papel do `n_estimators`

É a quantidade de árvores na floresta. Poucas árvores: a votação ainda é instável. Muitas árvores: a votação se estabiliza.

Diferente de `max_depth` ou `K` — aqui, mais árvores quase nunca piora, só fica mais lento pra treinar.



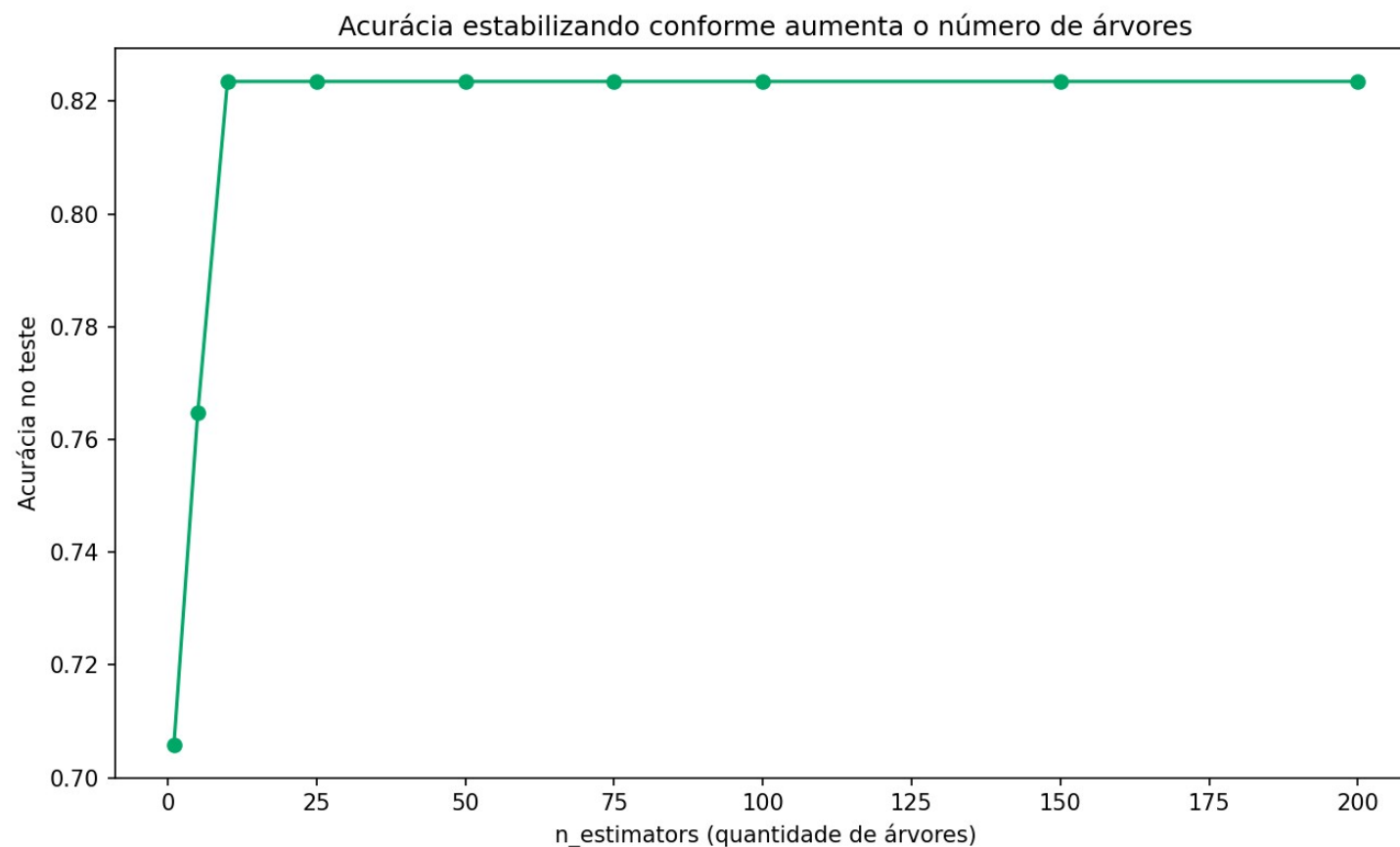
Vamos ver na prática

```
valores_n = [1, 5, 10, 25, 50, 75, 100, 150, 200]
acuracias = []
for n in valores_n:
    m = RandomForestClassifier(n_estimators=n, random_state=42)
    m.fit(X_train, y_train)
    acuracias.append(accuracy_score(y_test, m.predict(X_test)))

plt.plot(valores_n, acuracias, marker="o")
plt.show()
```



A acurácia se estabilizando



O fluxo no Scikit-Learn

Mesma gramática de sempre: `RandomForestClassifier(n_estimators=N)`, `fit()`, `predict()` — já bem familiar a essa altura.

Combina a simplicidade da árvore com a robustez de votar várias vezes.



Vamos ver na prática

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=5, stratify=y
)

modelo_final = RandomForestClassifier(n_estimators=100, random_state=42)
modelo_final.fit(X_train, y_train)

pred = modelo_final.predict(X_test)
print(f"Acuracia: {accuracy_score(y_test, pred):.2%}")
```



Importância das features no Random Forest

Mesma ideia da árvore única (`feature_importances_`), mas mais confiável — é uma média entre centenas de árvores, não o julgamento de uma só.

Repare: na árvore única, `bota_ovos` dominava sozinha (0,71). No Random Forest, o crédito fica mais distribuído entre várias features.

Isso acontece porque `max_features` às vezes impede uma árvore de usar `bota_ovos` primeiro, dando chance a outras features de aparecerem.

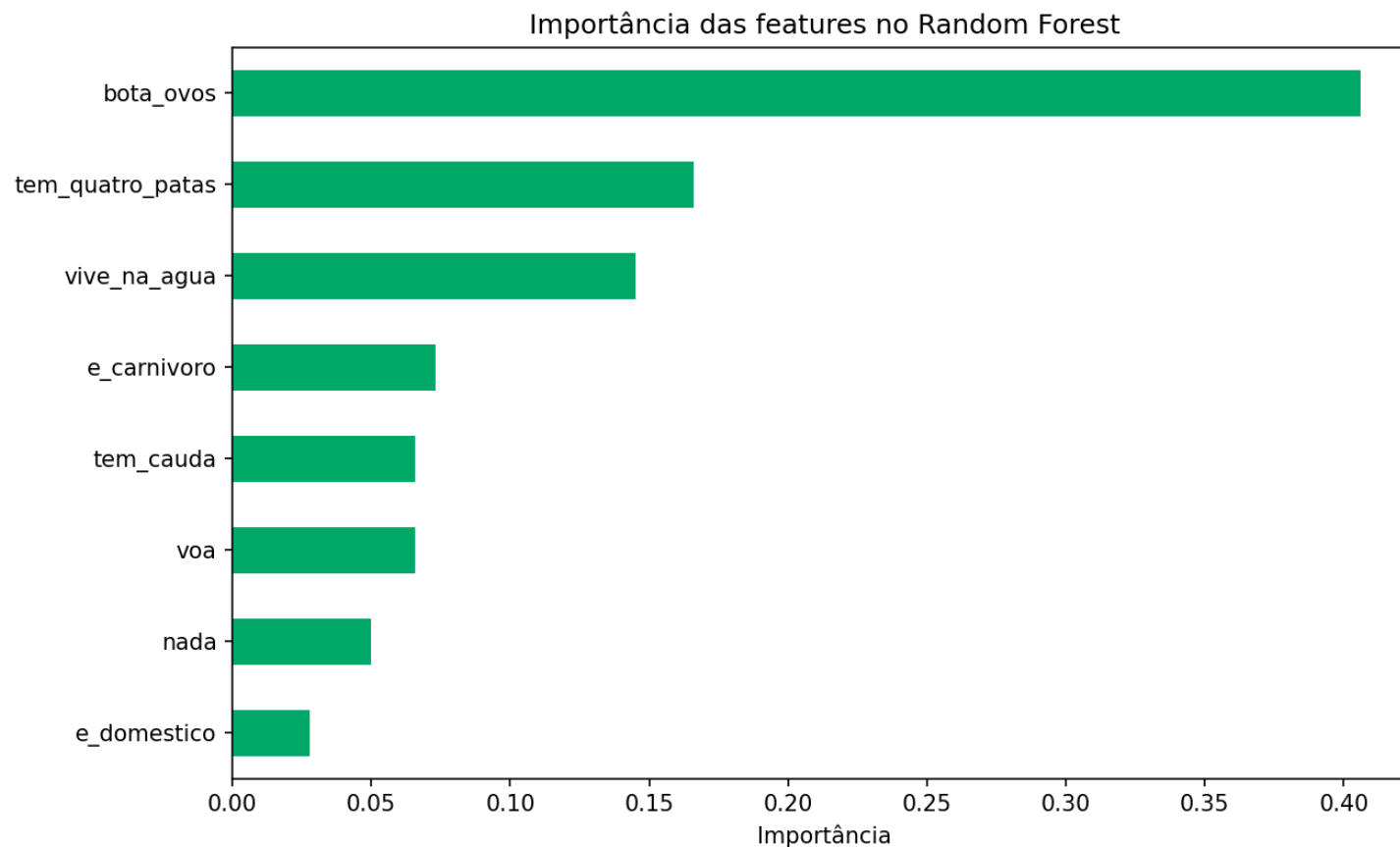


Vamos ver na prática

```
importancias = pd.Series(  
    modelo_final.feature_importances_, index=features  
)  
.sort_values()  
  
importancias.plot(kind="barh")  
plt.show()
```



Crédito mais distribuído entre as features



Muitas opiniões, uma decisão mais estável

- 1 **Votação resolve instabilidade** — Árvores individuais discordam; a média estabiliza
- 2 **Duas fontes de aleatoriedade** — Bootstrap (dados) + max_features (colunas)
- 3 **n_estimators quase nunca piora** — Diferente de max_depth ou K, mais árvore é mais seguro

A seguir: métricas completas (precisão, recall, F1) e validação cruzada — com o mpg de novo.

